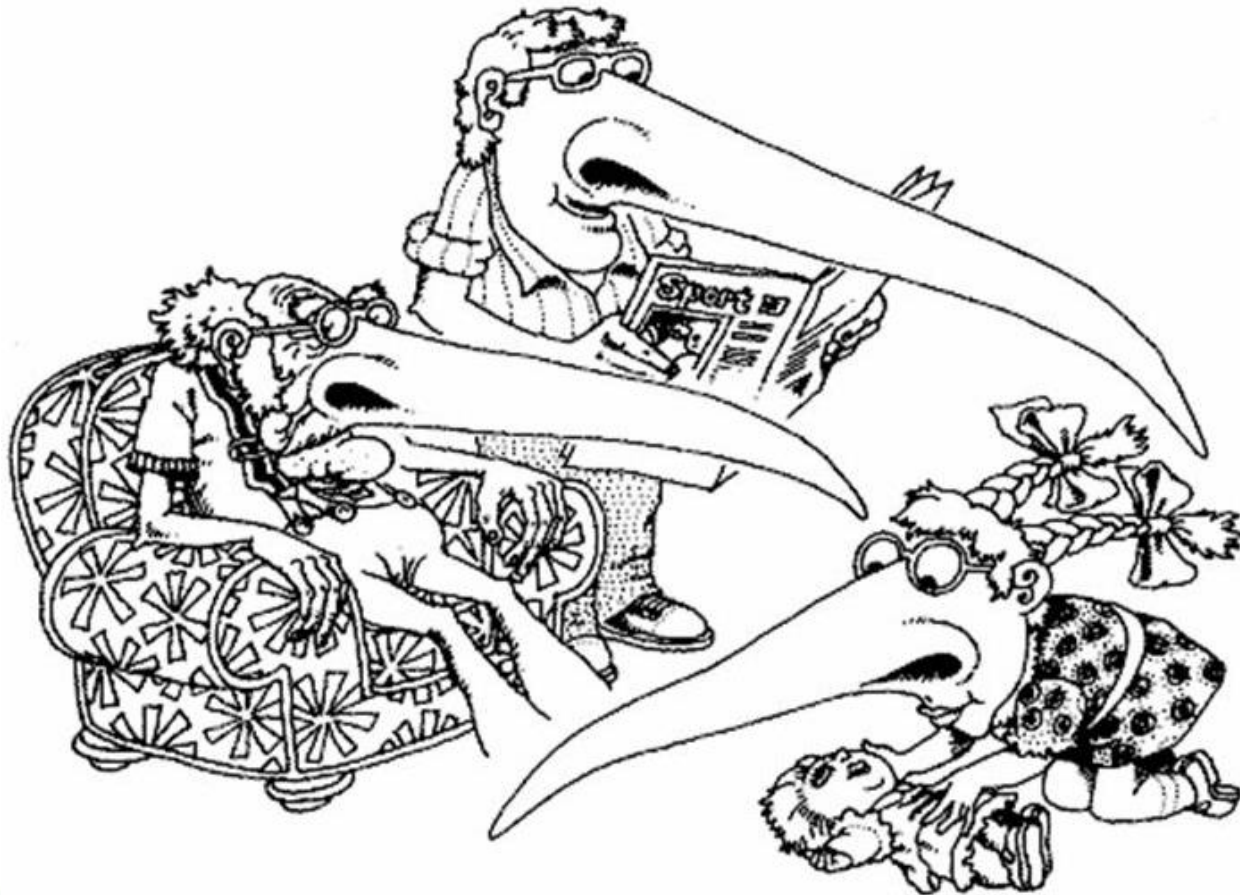


Pemrograman Berorientasi Objek



Inheritance

What is Inheritance?



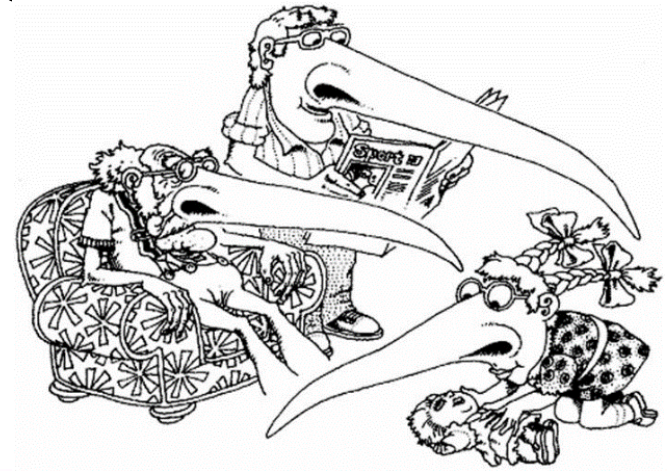
Inheritance

- ▶ The ability of a java class to 'pass down' all or some of their fields, attributes, and methods to another class that will be referred to as their child class
- ▶ Child class will be able to recognize the inherited fields and methods, and can use it without declaring or defining again



Inheritance

- Inheritance is an implementation of generalization-specialization class relationship or "is-a" relationship
- In short : it's the mechanism by which one class acquires the properties of another class



Inheritance

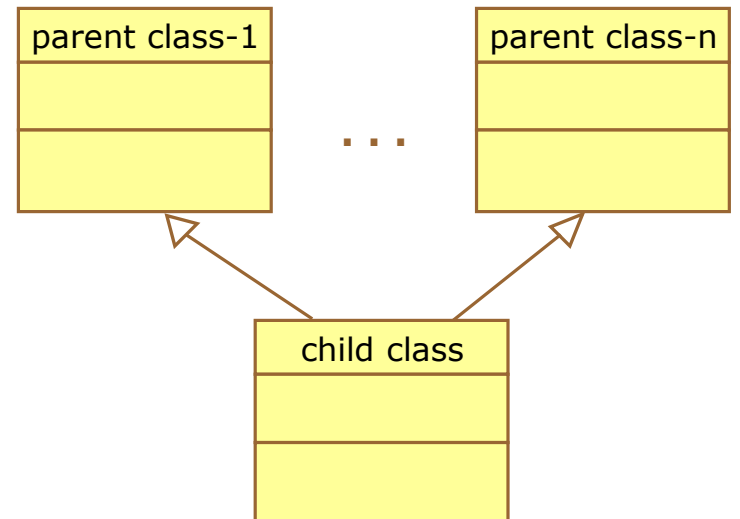
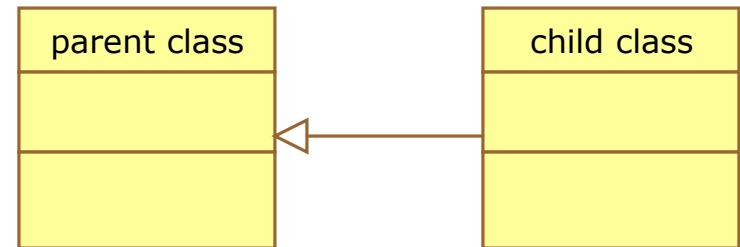
- Denoted by arrows (hollow and triangular head)



- The classes that passed down the attributes and methods are called
 - parent class, super class, or base class
- The classes that inherit the attributes and methods are called
 - Child class, subclass, or derived class

Types of Inheritance

- ▶ **Single Inheritance**
 - Child class inherit from 1 parent class
- ▶ **Multiple Inheritance**
 - Child class inherit from many parent classes



Java Class Inheritance

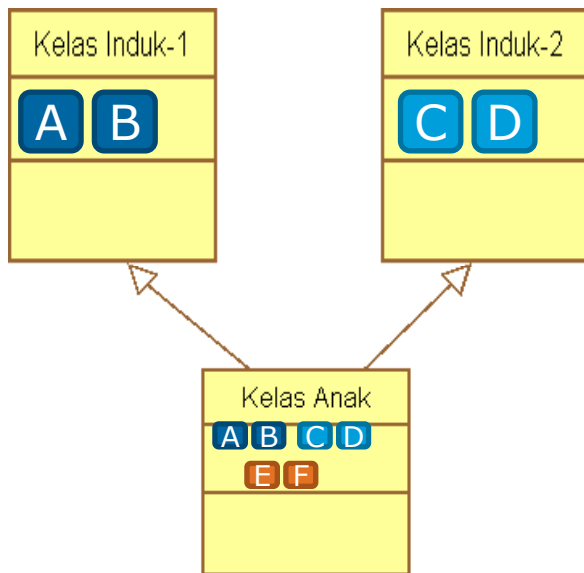
- ▶ Single inheritance only
- ▶ Using keyword extends

```
class ChildClass extends ParentClass{  
    // child class attributes  
    // child class methods  
}
```

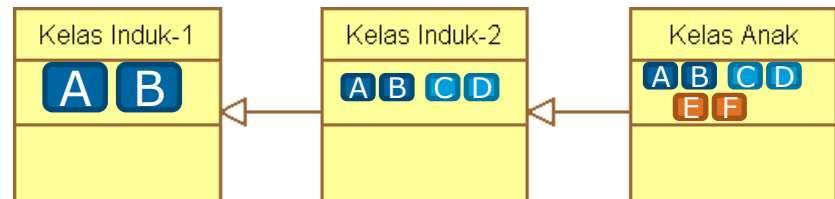
- ▶ All non-private attributes and methods are passed on
- ▶ To access the properties of parent
 - Use "super" keyword

Multiple Inheritance in Java

- ▶ Multiple Inheritance is handled using staged single inheritance



Base Multiple Inheritance



Multiple Inheritance using
Staged Single Inheritance

Example - Method Inheritance

```
public class Parent{  
    public String methodParent(){  
        return "this is method Parent";  
    }  
}
```

```
public class Child extends Parent{  
    public String methodChild(){  
        return "this is method Child";  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
        Parent p = new Parent();  
        Child c = new Child();  
  
        System.out.println(p.methodParent());  
        System.out.println(c.methodParent());  
  
        System.out.println(c.methodChild());  
        System.out.println(p.methodChild());  
    }  
}
```

```
> This is method Parent  
> This is method Parent  
  
> This is method Child  
> error: cannot find symbol
```

Child can access
public method of
Parent, but not vice
versa

Example – Variable Inheritance

```
public class Parent{  
    public int publicInt;  
    private int privateInt;  
    int defaultInt;  
}
```

```
public class Child extends Parent{  
    public void methodChild(){  
        publicInt = 10;  
        privateInt = 20;  
        defaultInt = 30;  
    }  
}
```

error: privateInt has
private access



Example

```
public class Parent{  
    public int publicInt;  
    private int privateInt;  
    int defaultInt;  
  
    public void setPrivateInt(int privateInt){  
        this.privateInt = privateInt;  
    }  
}
```

```
public class Child extends Parent{  
    public void methodChild(){  
        publicInt = 10;  
        setPrivateInt(20);  
        defaultInt = 30;  
    }  
}
```

Example

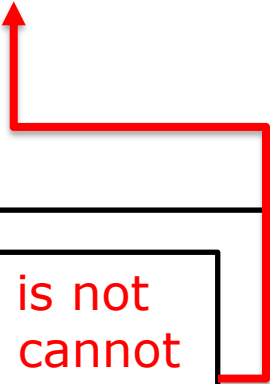
```
package test;

public class Parent{
    public int publicInt;
    private int privateInt;
    int defaultInt;

    public void setPrivateInt(int privateInt){
        this.privateInt = privateInt;
    }
}
```

```
import test.Parent;

public class Child extends Parent{
    public void methodChild(){
        publicInt = 10;
        setPrivateInt(20);
        defaultInt = 30;
    }
}
```



error: defaultInt is not
public in Parent; cannot
be accessed from
outside package

Access Modifier Protected

- ▶ All class in the same package
- ▶ All subclass (child class) even in different package



Example

```
package test;

public class Parent{
    public int publicInt;
    private int privateInt;
    int defaultInt;
    protected int protectedInt;

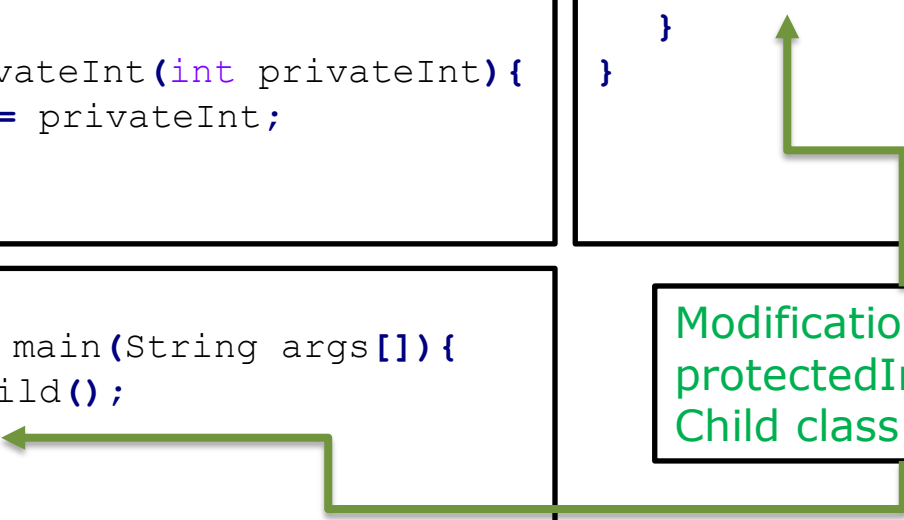
    public void setPrivateInt(int privateInt){
        this.privateInt = privateInt;
    }
}
```

```
import test.Parent;

public class Child extends Parent{
    public void methodChild(){
        publicInt = 10;
        setPrivateInt(20);
        protectedInt = 30;
    }
}
```

```
public class Driver{
    public static void main(String args[]){
        Child c = new Child();
        c.methodChild();
    }
}
```

Modification to
protectedInt done by
Child class



Example

```
package test;

public class Parent{
    public int publicInt;
    private int privateInt;
    int defaultInt;
    protected int protectedInt;

    public void setPrivateInt(int privateInt){
        this.privateInt = privateInt;
    }
}
```

```
import test.Parent;

public class Child extends Parent{

}
```

```
public class Driver{
    public static void main(String args[]){
        Child c = new Child();
        c.publicInt = 10;
        c.setPrivateInt(20);
        c.protectedInt = 40;
    }
}
```

error: protectedInt has
protected access in parent

(protectedInt is owned by
Parent in different
package)



Overriding

- Redefining field or method inherited by Parent class in Child class
- subclass can implement a parent class method more specifically based on its requirement.
- When overriding a field or method, the Child class will have access to both the original parent field/method and the new redefined child field/method
- Call parent's field/method using keyword "super"
 - Keyword "super" only available inside the child class

Example

```
public class Parent{  
    public String toString(){  
        return "this is method Parent";  
    }  
}
```

```
public class Child extends Parent{  
  
}
```

```
public class Driver{  
    public static void main(String args[]){  
        Child c = new Child();  
        Parent p = new Parent();  
  
        System.out.println(p.toString());  
        System.out.println(c.toString());  
    }  
}
```

```
> this is method Parent  
> this is method Parent
```

Example

```
public class Parent{  
    public String toString(){  
        return "this is method Parent";  
    }  
}
```

```
public class Child extends Parent{  
    public String toString(){  
        return "this method overridden  
        by Child";  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
        Child c = new Child();  
        Parent p = new Parent();  
  
        System.out.println(p.toString());  
        System.out.println(c.toString());  
    }  
}
```

> this is method Parent
> this method overridden by Child

Example

```
public class Parent{  
    public String toString(){  
        return "this is method Parent";  
    }  
}
```

```
public class Child extends Parent{  
    public String toString(){  
        return "this method overridden  
        by Child";  
    }  
    public String toStringParent(){  
        return super.toString();  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
        Child c = new Child();  
        Parent p = new Parent();  
  
        System.out.println(p.toString());  
        System.out.println(c.toString());  
        System.out.println(c.toStringParent());  
    }  
}
```

```
> this is method Parent  
> this method overridden by Child  
> this is method Parent
```

Example

```
public class Parent{  
    protected int number;  
  
}
```

```
public class Child extends Parent{  
    private int number;  
  
    public void methodChild(){  
        number = 10;  
        super.number = 20;  
  
        System.out.println(number);  
        System.out.println(super.number);  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
        Child c = new Child();  
        c.methodChild();  
    }  
}
```

```
> 10  
> 20
```

Example

```
public class Parent{
    protected int number;

    public void setNumber(int number){
        this.number = number;
    }

    public int getNumber(){
        return number;
    }
}
```

```
public class Child extends Parent{
    private int number;

    public void methodChild(){
        number = 10;
        super.number = 20;
    }
}
```

```
public class Driver{
    public static void main(String args[]){
        Child c = new Child();
        c.methodChild();

        System.out.println(c.getNumber());
    }
}
```

> 20

setNumber() and
getNumber() is owned by
parent, accessing number
value from parent (super)



Example

```
public class Parent{
    protected int number;

    public void setNumber(int number){
        this.number = number;
    }

    public int getNumber(){
        return number;
    }
}
```

```
public class Child extends Parent{
    private int number;

    public void methodChild(){
        number = 10;
        super.number = 20;
    }

    public int getNumber(){
        return number;
    }
}
```

```
public class Driver{
    public static void main(String args[]){
        Child c = new Child();
        c.methodChild();

        System.out.println(c.getNumber());
    }
}
```

> 10

If it's overridden,
the method will
access child field
first



Non-Access Modifier Final

- To limit the access to modify the class, field, or method
- Final modifier in method
 - Method cannot be overridden by child class, for use only
 - `public final void finMethod(){ ... }`

```
public class Parent{  
    public void publicMethod(){  
  
    }  
    final public void finMethod(){  
  
    }  
}
```

```
public class Child extends Parent{  
    public void publicMethod(){  
        //ok  
    }  
    public void finMethod(){  
        // forbidden  
    }  
}
```

Non-Access Modifier Final

- ▶ Final modifier in class
 - Class cannot be extended (subclassed) by other class
 - public final class FinClass{ ... }

```
Public final class FinalParent{  
  
}
```

```
public class Child extends FinalParent{  
    // forbidden  
}
```

Non-Access Modifier Final

► Final modifier in field/variable

- Field/variable is constant, value set when declared or initiated in constructors, and cannot be changed afterward
- private final double finDouble

```
public class Driver{  
    public static void main(String args[]){  
        final int a = 5;  
        final int b;  
        b = 8;  
  
        a = 15;    // forbidden  
        b = 21;    // forbidden  
    }  
}
```

- in class diagram, final constant is represented in all uppercase and using underscore to separate words

Non-Access Modifier Final

- ▶ Final modifier in Reference Variable
 - Once the variable refers to an object, the variable cannot be re-bound to reference another object.
 - But the object that it references is still mutable, if it was originally mutable.
 - Example:
 - `private final ClassName finObject;`

Non-Access Modifier Final

```
public class Parent{
    int number;

    public void setNumber(int number){
        this.number = number;
    }

    public int getNumber(){
        return number;
    }
}
```

```
public class Driver{
    public static void main(String args[]){
        Parent p = new Parent();
        final Parent fp1 = new Parent();
        final Parent fp2;
        fp2 = p;

        fp1 = p;           // forbidden
        fp2 = new Parent(); // forbidden

        fp2.setNumber(5);
        fp1.setNumber(20);
    }
}
```

Constructor **is not** inherited

- If there is only non-default constructor(s) in parent class
 - There must be at least one child constructor that invokes one parent constructor
 - Every constructors in child class must invoke one parent constructor
 - Invoke parent constructor using method `super(parameter)`
- If there is a default constructor in parent class
 - the child class does not have to invoke the parent constructor

Example

```
public class Parent{  
    protected int vP1;  
    protected double vP2;  
  
    public Parent(){  
    }  
  
    public void methodParent(){  
    }  
}
```

```
public class Child extends Parent{  
    private int vC;  
  
    public Child(int vC){  
        this.vC = vC;  
    }  
  
    public void methodChild(){  
    }  
}
```

If there is a default constructor or no constructor at all present at parent class, then child class does not have to invoke parent class constructor

child class may or may not have its own constructor

Example

```
public class Parent{
    protected int vP1;
    protected double vP2;

    public Parent(int vP1){
        this.vP1 = vP1;
    }

    public Parent(int vP1, double vP2 ){
        this.vP1 = vP1;
        this.vP2 = vP2;
    }

    public Parent(double vP2 ){
        this.vP2 = vP2;
    }

    public void methodParent(){
    }
}
```

```
public class Child extends Parent{
    private int vC;

    public Child(int vC, double vP){
        this.vC = vC;
        super(vP);
    }

    public void methodChild(){
    }
}
```

If there are non-default constructors in parent class, There must be AT LEAST one child constructor that invokes one parent constructor

Use keyword `super()` to invoke parent constructor

Example

```
public class Parent{
    protected int vP1;
    protected double vP2;

    public Parent(int vP1){
        this.vP1 = vP1;
    }

    public Parent(int vP1, double vP2 ){
        this.vP1 = vP1;
        this.vP2 = vP2;
    }

    public Parent(double vP2 ){
        this.vP2 = vP2;
    }

    public void methodParent(){
    }
}
```

```
public class Child extends Parent{
    private int vC;

    public Child(int vC, double vP){
        this.vC = vC;
        super(vP);
    }

    public Child(double vP){
        super(5, vP);
    }

    public Child(){
        super(2.5);
    }

    public void methodChild(){
    }
}
```

All child constructor must
invoke parent constructor

Example

```
public class Parent{
    protected int vP1;
    protected double vP2;

    public Parent(){
        vP1 = 50;
    }

    public Parent(int vP1, double vP2 ){
        this.vP1 = vP1;
        this.vP2 = vP2;
    }

    public Parent(double vP2 ){
        this.vP2 = vP2;
    }

    public void met
}
}
```

If parent class has default constructor, then the child class may or may not invoke the parent constructor at all

```
public class Child extends Parent{
    private int vC;

    public Child(int vC, double vP){
        super(vC, vP);
    }

    public Child(int vC){
        this.vC = vC;
    }

    public Child(){
        this.vC = 25;
    }

    public
}
```

Child constructors that did not invoke any parent constructor will automatically invoke the default parent constructor

When to Use

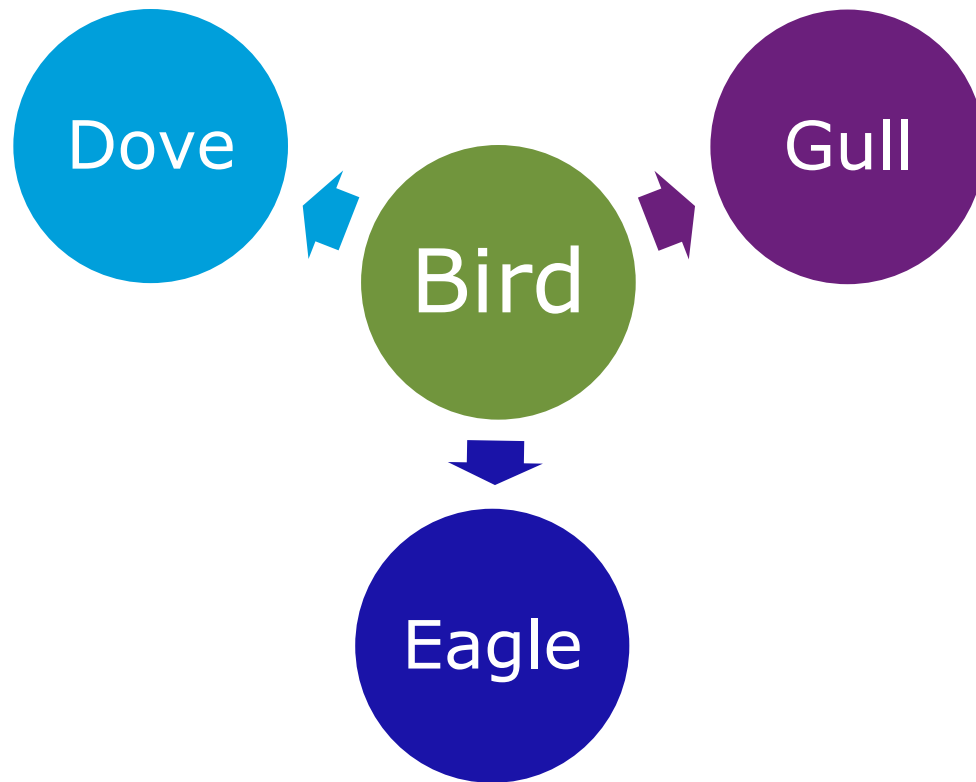
- ▶ Specialization
 - Top-down view
- ▶ Generalization
 - Bottom-up view

Specialization

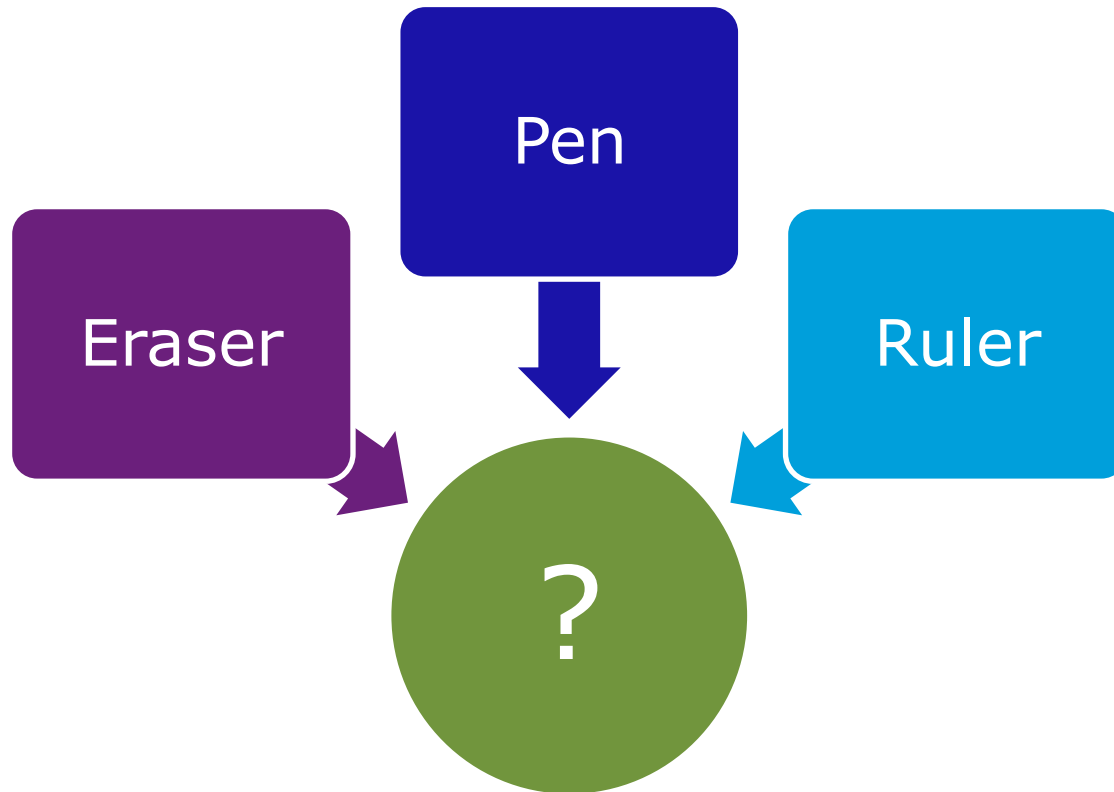


Bird

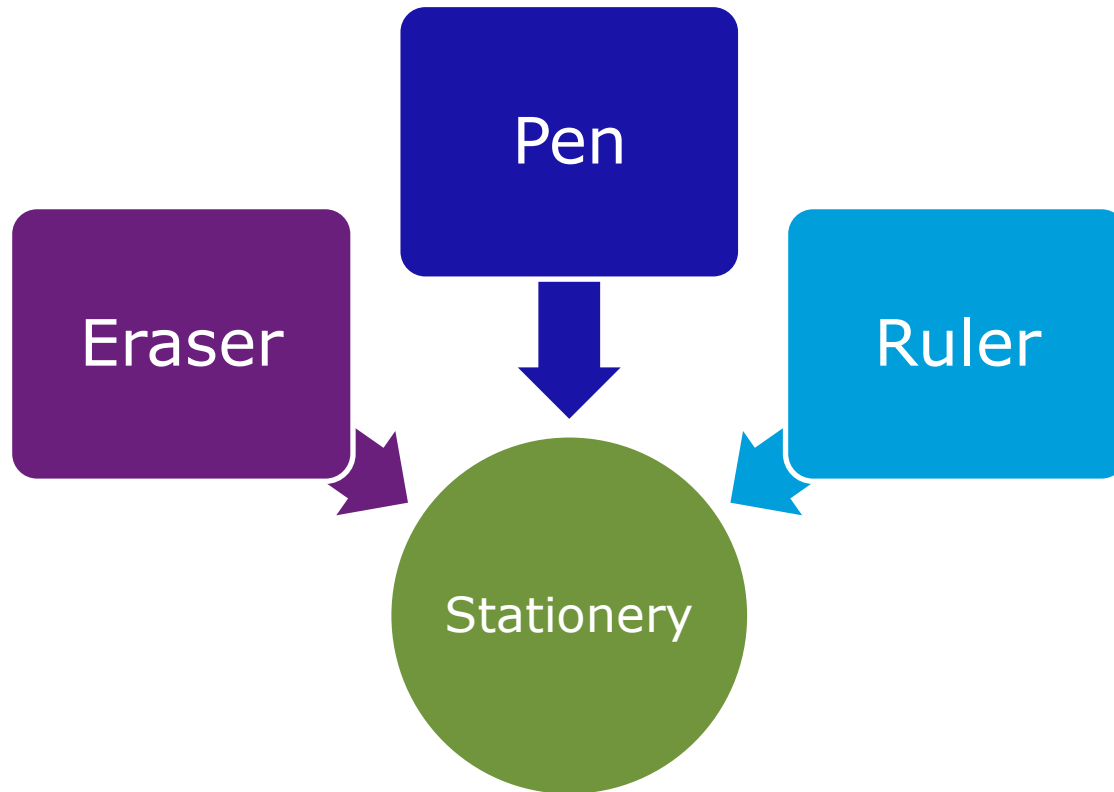
Specialization



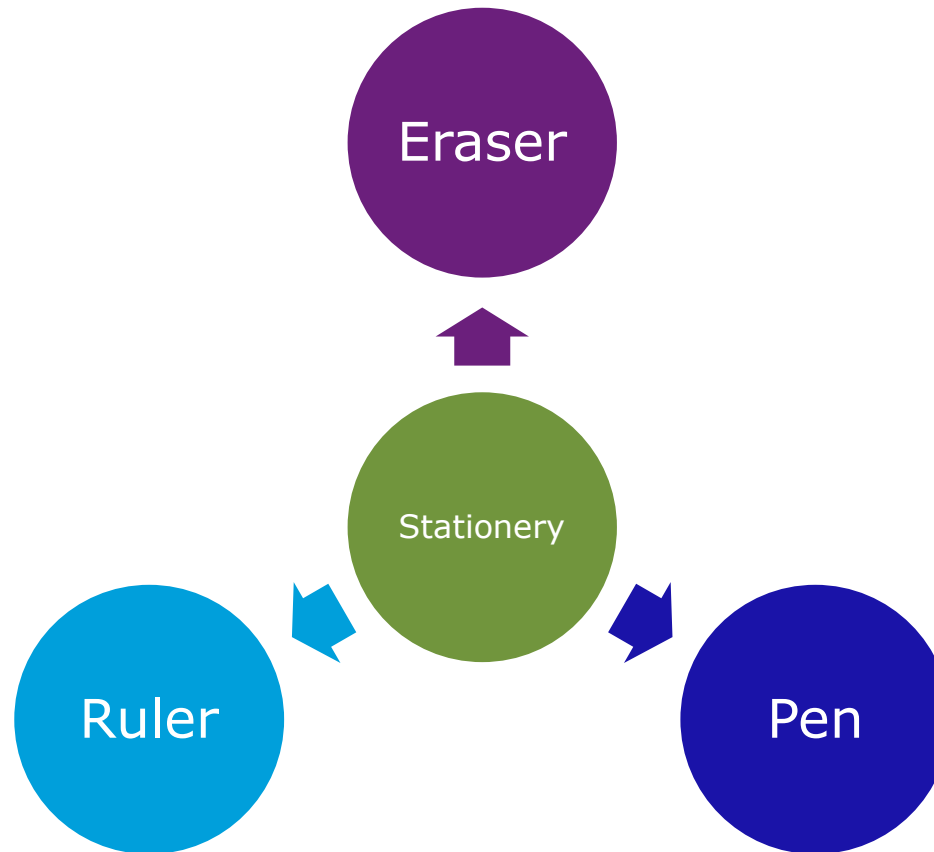
Generalization



Generalization

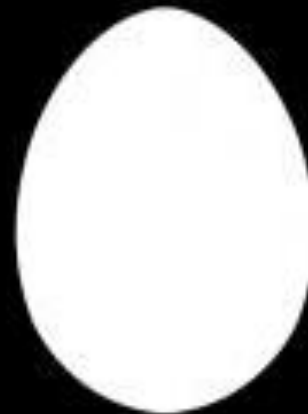
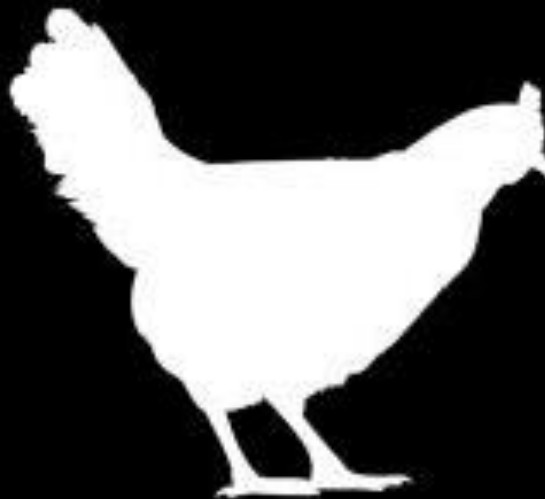


Specialization or Generalization ?





Fakultas Informatika
School of Computing
Telkom University



When to use

▶ Top-Down View

- to create a new classes with some behavior that already exists in other class (or similar with other class)
- to create more specific classes or modification from a base class
- to break down a large class to more specific classes
- to create a form of class hierarchy

▶ For those reasons we can create subclasses that inherit the base class

When to use

▶ Bottom-Up View

- When there several classes that generally the same (many methods and fields are the same or similar) but not necessary that one of the class is to be the child of another
 - To create a module to interacts between classes
- ### ▶ For those reasons we can create a parent class that collects the other classes as its child classes

Benefit of Inheritance

- ▶ Reusability
 - Reuse the methods and data of the existing class
- ▶ Extends
 - Adding new data and new methods to the existing class
- ▶ Modify
 - Modify the existing class by overriding its methods with your own implementation
- ▶ Reduction (Grouping and Organizing)

5 things you might find in an *Inheritance Hierarchy*:

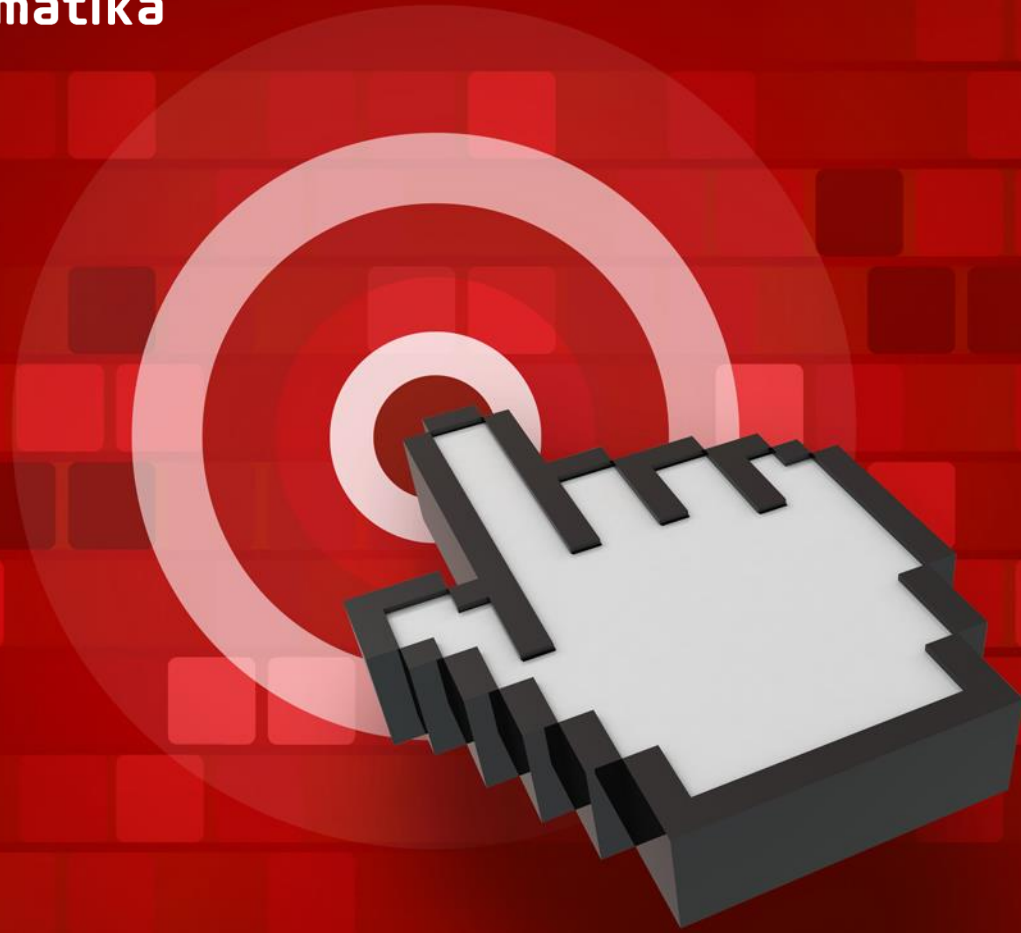
- superclass is too general to declare all behavior, so each subclass adds its own behavior
- superclass legislates an abstract behavior and therefore delegates implementation to its subclasses
- superclass specifies behavior, subclasses inherit behavior
- superclass specifies behavior, subclasses can choose to override behavior completely
 - just because a subclass inherits a method doesn't mean that it must act in the same way as its superclass
 - subclass can choose to reject its superclass' implementation of any method and "do it my way"
- superclass specifies behavior, subclasses can choose to override behavior in part
 - called partial overriding

Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU